Efrei Paris

Département de Mathématiques

Promotion 2028

Projet de Recherche Opérationnelle

Étude de la Complexité

Réalisé par :

Arthur	**Boutelier**
Baptiste	**Bertrand**
Thomas	**Bottalico**
Benjamin	**Binisti**
Valentin	**Belougne**

Année 2025/2026

1 Introduction et méthodologie

L'objectif de ce rapport est d'analyser et de comparer la complexité des algorithmes que nous avons implémentés pour résoudre le problème de transport. On va chercher à comprendre comment les temps d'exécution évoluent quand la taille du problème augmente et quel enchaînement d'algorithmes est le plus efficace.

On étudie quatre portions de code. D'abord les deux algorithmes qui produisent une proposition initiale. Nord-Ouest (temps noté θ_{NO}) et Balas-Hammer (temps noté θ_{BH}). Ensuite, la méthode du marche-pied avec potentiel appliquée à chacune de ces propositions que l'on notera t_{NO} et t_{BH}. Le but final est de comparer les deux chaînes complètes $(\theta_{NO} + t_{NO})$ et $(\theta_{BH} + t_{BH})$ pour savoir laquelle est la plus rapide.

Pour avoir des mesures représentatives, on génère des problèmes de transport aléatoires de taille $n \times n$. Les coûts $a_{i,j}$ sont des entiers tirés aléatoirement dans $\{1, \ldots, 100\}$. Les provisions P_i et commandes C_j sont construites à partir d'une matrice temporaire $\text{temp}_{i,j}$:

$$P_i = \sum_{j=1}^{n} \text{temp}_{i,j}, \qquad C_j = \sum_{i=1}^{n} \text{temp}_{i,j}$$

ce qui garantit automatiquement l'équilibre $\sum_i P_i = \sum_j C_j$, condition nécessaire pour appliquer le marche-pied. Les temps sont mesurés avec `time.perf_counter()` en Python. Pour chaque taille $n \in \{10,\ 40,\ 100,\ 400,\ 1000,\ 4000,\ 10000\}$, on effectue **100 répétitions** avec des données différentes afin d'obtenir un nuage de points représentatif. On note que pour $n \geq 4000$, t_{NO} n'a pas été mesuré car les temps estimés étaient trop élevés pour rester raisonnables sur une seule machine.

2 Résultats expérimentaux

2.1 Temps d'exécution mesurés

Table 1 – Temps moyens et maximaux (en secondes) sur 100 répétitions

2*n	θ_{NO} (s)		θ_{BH} (s)		t_{NO} (s)		t_{BH} (s)	
	moy	max	moy	max	moy	max	moy	max
10	$< 10^{-4}$	10^{-4}	$2{\cdot}10^{-4}$	$4{\cdot}10^{-4}$	$6{\cdot}10^{-4}$	$9{\cdot}10^{-4}$	$3{\cdot}10^{-4}$	$6{\cdot}10^{-4}$
40	$< 10^{-4}$	$< 10^{-4}$	0,0014	0,0016	0,0183	0,0277	0,0052	0,0080
100	10^{-4}	10^{-4}	0,0082	0,0095	0,180	0,216	0,045	0,060
400	$3{\cdot}10^{-4}$	$4{\cdot}10^{-4}$	0,160	0,202	5,91	6,66	1,09	1,46
1 000	$7{\cdot}10^{-4}$	0,0012	1,40	2,69	45,8	76,0	4,82	10,4
4 000	0,003	0,004	48,6	50,8	—	—	61,8	135
10 000	0,009	0,021	492	772	—	—	420	899

On releve que θ_{NO} est extrêmement rapide. Pour $n = 10\,000$, θ_{NO} vaut $0,021$. θ_{BH} lui est déjà à 772 secondes pour le même nombre d'itération.. L'écart entre les deux est donc considérable. Cela ne signifie pas pour autant que Nord-Ouest est forcément meilleur que Balas-Hammer.

2.2 Qualité des solutions initiales

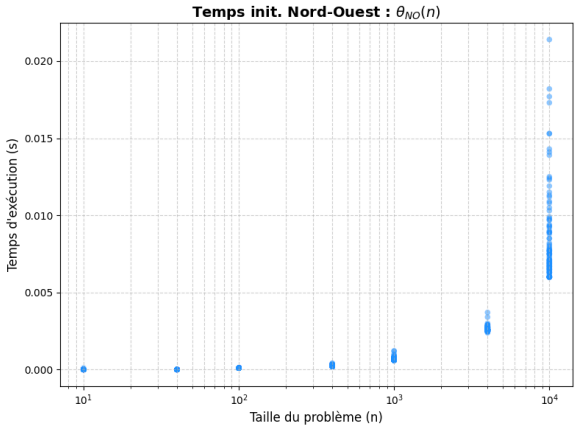
Table 2 – Coûts initiaux et finaux moyens – les deux méthodes convergent vers le même optimum

n	Coût init. NW	Coût init. BH	Coût final	Rapport NW / BH
10	249 493	106 482	78 050	2,3×
40	4 067 312	597 352	368 455	6,8×
100	25 574 283	1 921 402	1 097 059	13,3×
400	407 180 193	12 737 010	8 451 802	32,0×
1 000	$2{,}55 \times 10^9$	$6{,}24 \times 10^7$	$5{,}05 \times 10^7$	40,8×

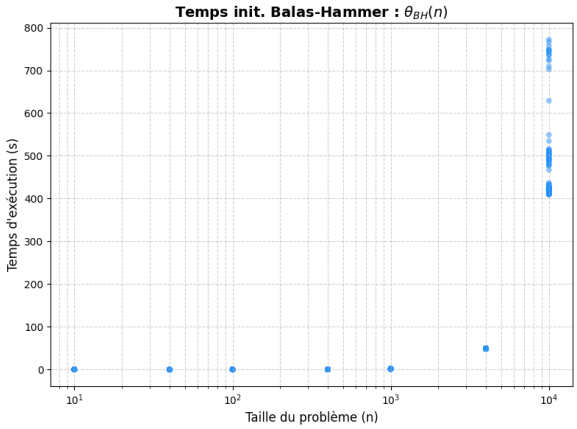
On voit que les deux méthodes aboutissent exactement au même coût optimal (colonne "Coût final"), ce qui confirme que notre marche-pied converge correctement vers l'optimum global, quelle que soit la proposition de départ.

En revanche, la qualité de la proposition initiale est radicalement différente. Pour $n = 10$, le coût initial de Nord-Ouest est 2,3 fois plus élevé que celui de Hammer. Pour $n = 1\,000$, ce rapport est de 40,8. Autrement dit, plus le problème est grand, plus la proposition Nord-Ouest est proportionnellement mauvaise. C'est ce phénomène qui explique que le marche-pied doit faire beaucoup plus d'itérations quand il part d'une proposition Nord-Ouest.

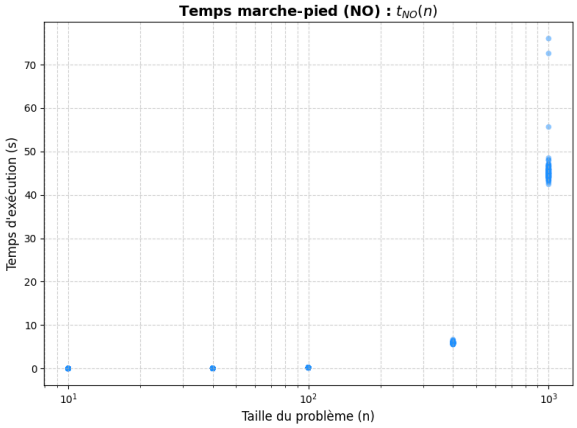
2.3 Nuages de points



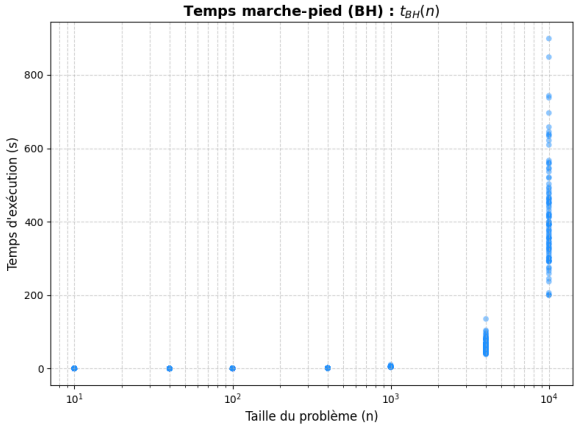
(a) Algorithme Nord-Ouest : $\theta_{NO}(n)$



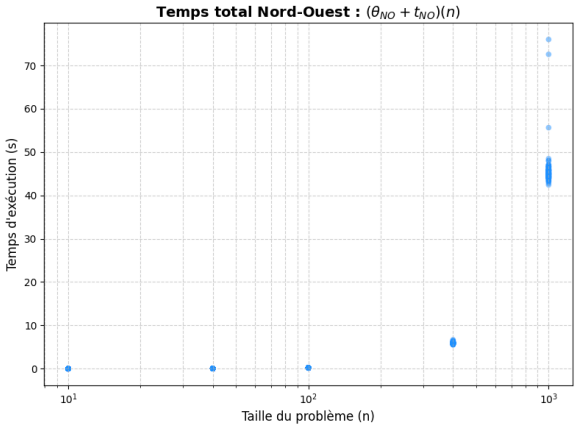
(b) Algorithme Balas-Hammer : $\theta_{BH}(n)$



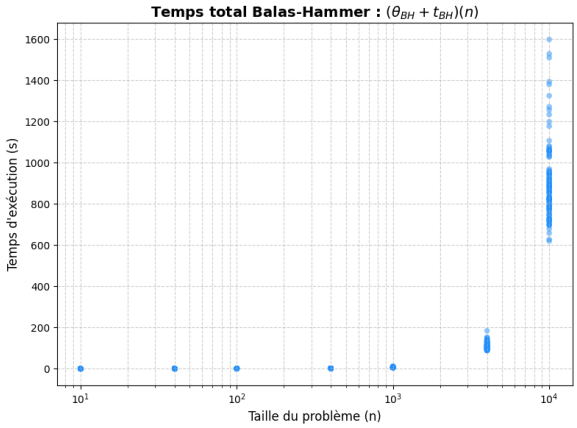
(c) Marche-pied après NO : $t_{NO}(n)$



(d) Marche-pied après BH : $t_{BH}(n)$



(e) Temps total NO : $(\theta_{NO} + t_{NO})(n)$



(f) Temps total BH : $(\theta_{BH} + t_{BH})(n)$

Nuages de points des temps d'exécution en fonction de la taille n du problème (echelle logarithmique)

3 Analyse de la complexité dans le pire des cas

Pour chaque valeur de n, on retient la valeur maximale parmi les 100 mesures. C'est cette valeur supérieure qui représente la complexité dans le pire des cas. On la compare ensuite aux références

classiques pour identifier le type de croissance.

3.1 θ_{NO} – Nord-Ouest : pourquoi $O(n)$?

L'algorithme Nord-Ouest part de la case en haut à gauche du tableau et remplit les cases une par une, en avançant soit vers la droite soit vers le bas selon que c'est la provision ou la commande qui est épuisée en premier. Au total, il remplit exactement $n + m - 1 = 2n - 1$ cases (pour un problème carré), ce qui correspond à un unique parcours linéaire du tableau. Il n'y a aucune boucle imbriquée, aucun tri, aucun recalcul.

On s'attend donc à une complexité en $O(n)$. Les mesures le confirment. θ_{NO} reste inférieur à 10^{-4} s jusqu'à $n = 400$ et n'atteint que 0,021 s pour $n = 10\,000$. En passant de $n = 1\,000$ à $n = 10\,000$ (facteur 10). Le temps maximum est multiplié par environ 18, ce qui est proche du facteur théorique 10. L'écart étant dû aux effets système (ordonnanceur, cache).

$$\boxed{\theta_{NO}(n) \in O(n)}$$

3.2 θ_{BH} – Balas-Hammer : pourquoi $O(n^2 \log n)$?

Balas-Hammer est plus coûteux. Avant de démarrer les itérations, l'algorithme construit pour chaque ligne et chaque colonne une liste triée des coûts, afin de pouvoir retrouver efficacement les deux minimums. Ce tri initial s'effectue en $O(n \log n)$ pour chacune des $2n$ lignes/colonnes, soit un coût de départ en $O(n^2 \log n)$.

Ensuite, à chaque itération, la pénalité de chaque ligne et colonne encore disponible est obtenue en temps quasi-constant grâce à ces structures (tas via `heapq`). Le calcul de toutes les pénalités prend donc $O(n)$ par itération. On sélectionne ensuite la ligne ou colonne de pénalité maximale et on y affecte le minimum de coût. Cette opération est répétée jusqu'à ce que toutes les provisions et commandes soient satisfaites, soit $O(n)$ fois au total.

On a donc une phase d'initialisation en $O(n^2 \log n)$ et une boucle principale en $O(n) \times O(n) = O(n^2)$, ce qui donne une complexité globale dominée par $O(n^2 \log n)$. Les mesures le confirment. En passant de $n = 100$ à $n = 1\,000$, $\theta_{BH}^{\max}$ passe de 0,0095 s à 2,69 s, soit un facteur ≈ 283. Un $O(n^2)$ pur prédirait un facteur 100, et un $O(n^2 \log n)$ un facteur $100 \times \frac{\log 1000}{\log 100} \approx 150$. Le facteur 283 est supérieur à 150, ce qui s'explique par les effets de cache, les appels à recharger_heap lorsque le tas est épuisé, et la dégradation des accès mémoire sur les grandes matrices.

$$\boxed{\theta_{BH}(n) \in O(n^2 \log n)}$$

3.3 t_{NO} – Marche-pied depuis Nord-Ouest : pourquoi $O(n^2 \log n)$?

Le nombre d'itérations du marche-pied dépend directement de l'écart entre la proposition initiale et l'optimum. Or, comme le montre le tableau 2, cet écart est très important pour Nord-Ouest et il s'aggrave quand n augmente. Le rapport entre le coût initial NW et le coût optimal passe de 3,2× pour $n = 10$ à 50,5× pour $n = 1\,000$. Cela signifie que le marche-pied doit effectuer de plus en plus d'itérations pour "rattraper" le mauvais départ.

En pratique, entre $n = 100$ et $n = 400$ (facteur 4 sur n), $t_{NO}^{\max}$ est multiplié par ≈ 31. Un $O(n^2)$ pur donnerait un facteur $4^2 = 16$, et un $O(n^3)$ donnerait $4^3 = 64$. On est entre les deux, ce qui est cohérent avec un $O(n^2 \log n)$. Le facteur théorique $16 \times \log_4(4) \approx 16 \times 2 = 32$ est très proche

des 31 observés.

$$\boxed{t_{NO}(n) \in O(n^2 \log n)}$$

3.4 t_{BH} – Marche-pied depuis Balas-Hammer : pourquoi $O(n^2)$?

Contrairement à Nord-Ouest, la proposition de Balas-Hammer est déjà très proche de l'optimum. Le rapport entre le coût initial et le coût final ne dépasse pas 1,5 pour $n \leq 1\,000$, ce qui veut dire que le marche-pied n'a besoin que de quelques itérations pour converger. Le nombre d'itérations reste donc faible et ne croît pas aussi vite qu'avec Nord-Ouest.

Entre $n = 100$ et $n = 400$, $t_{BH}^{\max}$ est multiplié par ≈ 24, proche de $4^2 = 16$, ce qui confirme une croissance quadratique. Le léger dépassement par rapport au facteur théorique s'explique de la même façon que pour θ_{BH}, a savoir les effets de cache sur les grandes matrices.

$$\boxed{t_{BH}(n) \in O(n^2)}$$

4 Comparaison globale des deux approches

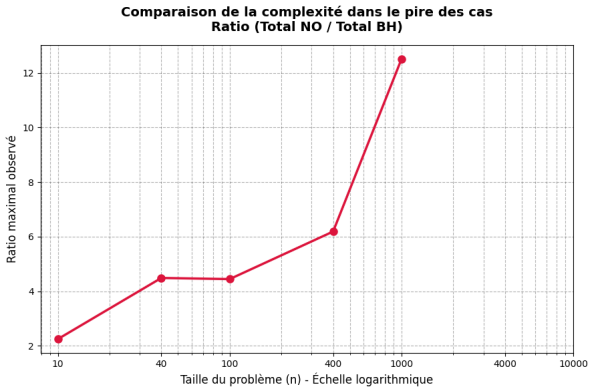


FIGURE 2 – Évolution du ratio $(t_{NO} + \theta_{NO})/(t_{BH} + \theta_{BH})$ selon n.

On observe que le ratio $\frac{t_{NO}+\theta_{NO}}{t_{BH}+\theta_{BH}}$ augmente avec n. Pour de petites valeurs de n, les deux algorithmes sont comparables, mais lorsque n augmente, l'algorithme de Nord-Ouest avec marche-pied devient significativement plus lent que Balas-Hammer avec marche-pied. Cela nous montre que plus les données sont grande plus l'algorithme de Nord-Ouest devient moins efficace et donc Balas-Hammer devient le plus adapté pour des gros jeu de données.

5 Conclusion

L'algorithme Nord-Ouest est le plus rapide à l'initialisation mais fournit une proposition initiale de très mauvaise qualité. Le rapport coût initial / oblige le marche-pied à un effort important. Balas-Hammer est plus coûteux à l'initialisation, mais sa proposition proche de l'optimum réduit le cout du marche-pied

La chaîne $(\theta_{BH} + t_{BH})$ se révèle nettement plus compétitive dès que n devient grand. Au final, bien que Nord-Ouest semble attrayant par sa rapidité d'exécution, son coût d'initialisation masque un surcoût rédhibitoire lors de l'optimisation. Pour tout problème de transport de taille significative, la chaîne Balas-Hammer + marche-pied est donc à privilégier.